MEMORIA
EJERCICIO EVALUABLE 3

Sistemas distribuidos

Grupo 80- Curso 25/26

Nombres, apellidos y NIA	
Alumno 1:	Daniel Santos Méndez (100522786) Grupo 80 - Grado en Ingeniería Informática
Alumno 2:	Sergio Aladro Municio (100522360) Grupo 80 - Grado en Ingeniería Informática

Parte 1. Arquitectura general del sistema

En este ejercicio hemos mantenido la misma base de las prácticas anteriores, pero cambiando la forma de comunicación para usar **ONC RPC**. Esto significa que **app-cliente.c** sigue utilizando las funciones de **claves.h** igual que antes, sin tener que preocuparse de cómo se envían las peticiones al servidor. La diferencia está en que ahora esas llamadas ya no se hacen por sockets programados manualmente, sino mediante RPC. Además, el nuevo archivo principal de esta práctica es **clavesRPC.x**, a partir del cual se generan automáticamente los archivos de soporte con rpcgen.

El sistema se organiza en tres capas claramente separadas. La primera es **app-cliente.c**, el cliente de prueba, que usa la interfaz de **claves.h** sin saber nada de cómo funciona la comunicación. La segunda es **libclaves.so**, la biblioteca que implementa esa interfaz haciendo llamadas RPC al servidor mediante **proxy-rpc.c**. La tercera es el servidor **clavesRPC_server**, que recibe las peticiones RPC y delega en **claves.c**, donde está la lógica real del servicio.

Esta separación en capas nos ha parecido la forma más limpia de organizar el proyecto, porque permite que **app-cliente.c** no tenga que cambiar nada respecto a las prácticas anteriores. El cliente sigue llamando a **destroy(), set_value(), get_value()** y demás como si todo fuera local, pero por debajo esas llamadas viajan por red hasta el servidor. Eso es exactamente lo que persigue RPC: hacer transparente la distribución.

Parte 2. Diseño de la interfaz RPC (clavesRPC.x)

El primer paso fue diseñar **clavesRPC.x**, que es el archivo que define qué operaciones ofrece el servidor y con qué tipos de datos trabaja. Para ello tuvimos que traducir la interfaz de **claves.h** a la sintaxis XDR que entiende rpcgen.

El aspecto más delicado fue representar los tipos de datos. Para las cadenas usamos string con límite explícito (255 caracteres), para el vector de floats usamos un typedef de array variable, y para la estructura Paquete creamos un struct equivalente llamado **paquete_rpc**. El caso más especial fue **get_value**: como necesita devolver varios valores a la vez y XDR solo permite un valor de retorno por procedimiento, definimos una estructura **rpc_get_value_res** que agrupa el código de estado, value1, **N_value2, V_value2 y value3**. Así, con una sola respuesta RPC el cliente recibe todo lo que necesita. Usamos las opciones -N y -M de rpcgen para poder pasar múltiples argumentos directamente en los procedimientos, sin tener que empaquetar todo en estructuras adicionales. Esto hace que la interfaz en clavesRPC.x sea mucho más legible y cercana a la definición original de claves.h.

Parte 3. Estructura de archivos e integración de rpcgen

El proyecto está organizado de forma que se distingue claramente qué archiv**os son nuestros y cuáles los genera rpcgen automáticamente. Los archivos propios son: claves.h, claves.c, app-cliente.c, clavesRPC.x, proxy-rpc.c, clavesRPC_server.c y app-cliente-concurrente.c**.
Los archivos generados por rpcgen (**clavesRPC.h, clavesRPC_clnt.c, clavesRPC_svc.c y clavesRPC_xdr.c**) no se incluyen en el repositorio y se regeneran con make rpcgen.

Decidimos separar la regla rpcgen en cuatro llamadas independientes en lugar de usar rpcgen -aNM, porque la opción -a genera también una plantilla de **clavesRPC_server.c** y la sobreescribe si ya existe. Al generar cada archivo por separado evitamos ese problema y el servidor implementado por nosotros nunca corre riesgo de ser borrado por error. La integración de todos estos archivos se ha hecho separando claramente la parte cliente y la parte servidor. En el cliente, la biblioteca libclaves.so se apoya en **proxy-rpc.c** y en los archivos generados por rpcgen para transformar las llamadas normales de claves.h en llamadas remotas al servidor. En el servidor, **clavesRPC_server.c** recibe las peticiones RPC y utiliza claves.c para ejecutar realmente las operaciones sobre las tuplas. De esta forma, los archivos generados automáticamente sirven como soporte de comunicación, mientras que la lógica real del sistema sigue estando en el código desarrollado por nosotros.

Parte 4. Compilación y ejecución

Para compilar el proyecto hay que ejecutar los siguientes pasos desde la raíz del directorio:

make rpcgen # genera clavesRPC.h, clavesRPC_clnt.c, clavesRPC_svc.c y clavesRPC_xdr.c
make # compila todo y genera libclaves.so, app-cliente y clavesRPC_server

Para ejecutar el sistema es necesario abrir dos terminales. En la primera se arranca el servidor, que no necesita ningún argumento porque el puerto lo gestiona rpcbind automáticamente:

./clavesRPC_server

En la segunda terminal se define la variable de entorno con la IP del servidor y se lanza el cliente:

export IP_TUPLAS=localhost ./app-cliente

Para limpiar los archivos compilados se puede usar **make clean**, y para eliminar también los archivos generados por rpcgen se usa **make distclean**.

Parte 5. Implementación de proxy-rpc.c

proxy-rpc.c es el núcleo de la parte cliente. Su función es implementar las seis operaciones de claves.h pero, en lugar de ejecutarlas localmente, hacer una llamada RPC al servidor por cada una. Para no repetir código en las seis funciones, extrajimos la lógica de conexión en una función auxiliar privada **crear_cliente()**, que lee la IP desde la variable de entorno **IP_TUPLAS** y crea el manejador RPC con **clnt_creat**e usando TCP.

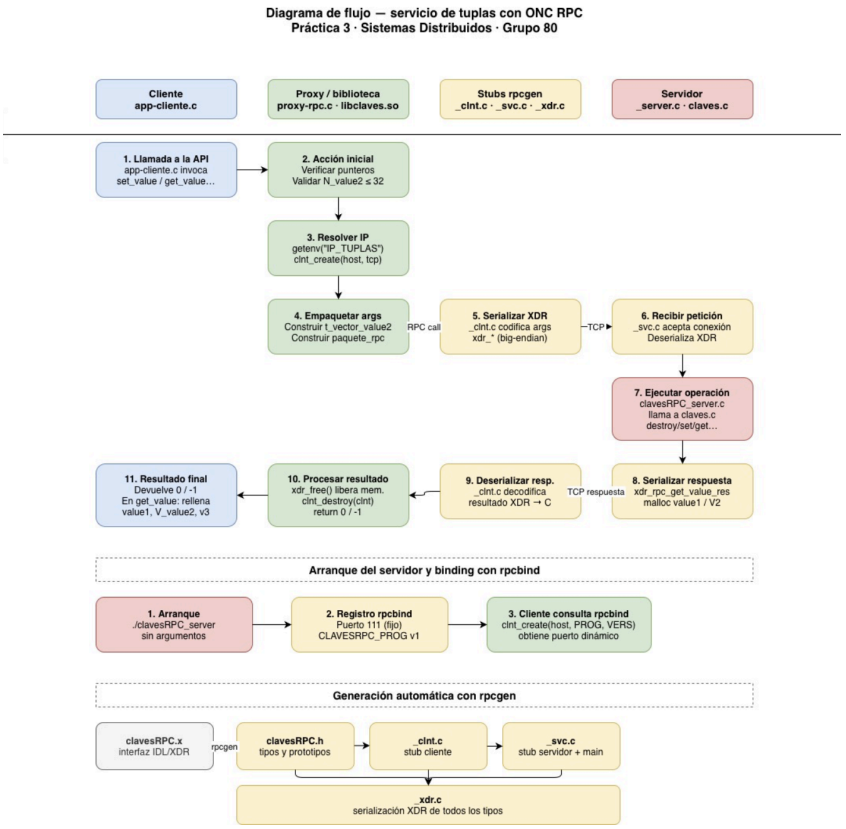
Cada función sigue el mismo patrón: crear el manejador, invocar el procedimiento remoto, destruir el manejador y devolver el resultado. Si en cualquier punto la llamada RPC falla (**clnt_create** devuelve NULL o el estado RPC no es **RPC_SUCCESS**), la función devuelve -1, cumpliendo con el contrato de claves.h que considera cualquier error de comunicación como un error del servicio. El caso más cuidadoso fue get_value: como la respuesta incluye memoria dinámica reservada por el servidor (**value1 y V_value2**), hay que llamar a **xdr_free()** para liberarla una vez copiados los datos al buffer del llamante. Esto se hace tanto en el caso de éxito como en el de error, para evitar fugas de memoria.

Parte 6. Implementación de clavesRPC_server.c

clavesRPC_server.c implementa los seis procedimientos remotos definidos en **clavesRPC.x**
En la mayoría de los casos la implementación es directa: los stubs de rpcgen ya se encargan de deserializar los argumentos y de serializar la respuesta, así que cada función solo tiene que hacer la conversión de tipos entre los tipos RPC y los tipos de claves.h, y llamar a la función correspondiente de **claves.c**. La función más elaborada es **rpc_get_value_1_svc**, que necesita reservar memoria dinámica para construir la respuesta. XDR requiere que los punteros en la respuesta sean válidos en el momento de la serialización, por lo que reservamos memoria con malloc para value1 y para el vector de floats. Esta memoria se libera automáticamente después de enviar la respuesta gracias a la función **clavesrpc_prog_1_freeresult**, que llama a **xdr_free** con la función de serialización correspondiente. También añadimos trazas en stderr para poder ver en tiempo real qué peticiones está atendiendo el servidor, lo que resulta útil tanto para depurar como para verificar el funcionamiento concurrente.

Parte 7. Diagrama de flujo

El flujo de una llamada RPC en este sistema sigue siempre el mismo esquema. Primero, **app-cliente.c** llama a una función de claves.h, por ejemplo set_value. Esa llamada llega a **proxy-rpc.c**, que lee **IP_TUPLAS**, crea el manejador RPC y llama al stub del cliente generado por rpcgen. El stub serializa los argumentos en formato XDR y los envía por TCP al servidor. En el servidor, el stub generado por rpcgen recibe el mensaje, deserializa los argumentos y llama a la función correspondiente en **clavesRPC_server.c**. Esta función llama a claves.c, obtiene el resultado, lo devuelve al stub del servidor, que lo serializa y lo envía de vuelta al cliente. Finalmente, el stub del cliente deserializa la respuesta y la devuelve a **proxy-rpc.c**, que la retorna a **app-cliente.c**.



Parte 8. Pruebas y validación

Se han realizado dos tipos de pruebas. La primera es la prueba funcional con **app-cliente**, que ejecuta 16 tests secuenciales cubriendo todos los casos: inserción correcta, duplicado (error esperado), **N_value2** fuera de rango, existencia, lectura, modificación, borrado y destroy. Todos los tests pasan correctamente.

La segunda es la prueba de concurrencia, realizada con **app-cliente-concurrente** y el script **test_concurrente.sh**. Este script lanza 5 instancias del cliente en paralelo, cada una trabajando con su propia clave (**clave_1 a clave_5**) para evitar interferencias entre clientes. El resultado fue que los 5 clientes completaron todos sus tests correctamente de forma simultánea, lo que confirma que el servidor gestiona bien el acceso concurrente al almacén de tuplas.

Parte 9. Conclusión del ejercicio

En general, esta práctica nos ha servido para entender mejor cómo se puede adaptar un servicio distribuido ya existente para que funcione mediante RPC. A diferencia de la práctica anterior, en esta no hemos tenido que diseñar manualmente toda la comunicación entre cliente y servidor, sino centrarnos más en definir bien la interfaz del servicio, integrar los archivos generados automáticamente y conectar todo con la lógica que ya teníamos implementada.

En conjunto, creemos que ha sido una práctica útil porque nos ha ayudado a ver de forma más clara cómo funciona la invocación remota de procedimientos y cómo se organiza un sistema distribuido cuando se trabaja con herramientas de más alto nivel. Además, también nos ha servido para reforzar la importancia de mantener una buena estructura del proyecto, separar bien las responsabilidades de cada parte y comprobar que todo funcione correctamente en el entorno de ejecución real.